LAPORAN PRAKTIKUM
TEKNOLOGI IOT II

Praktikum: 4.5.1

Latihan: 4.6

Dosen Pengampu:

1. Dr. Ir. Septafiansyah Dwi Putra - NIP : 198809112009121001
2. Agus Ambarwari, S.Pd., M.Kom. - NIP : 199208082019031020

Disusun oleh :

Ananda Haris Rahmadi (25782032)

PROGRAM STUDI TEKNOLOGI REKAYASA INTERNET

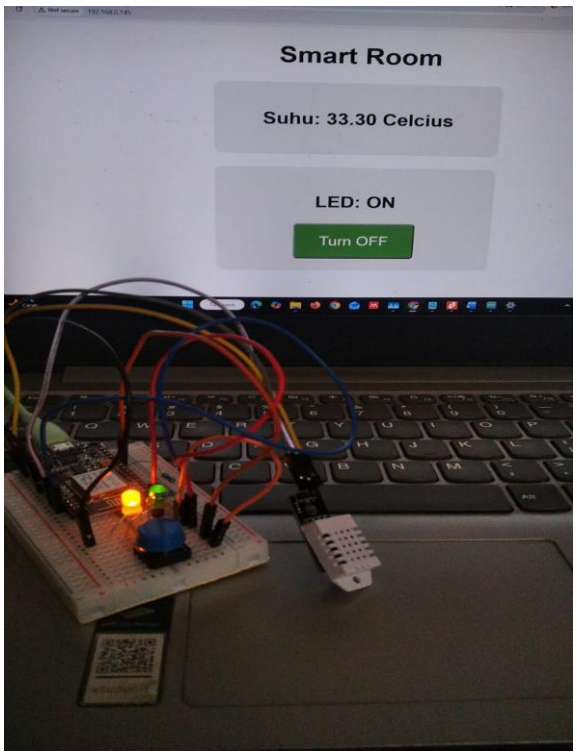
JURUSAN TEKNOLOGI INFORMASI

POLITEKNIK NEGERI LAMPUNG

2026

Praktikum: 4.5.1

Saat on



Saat off

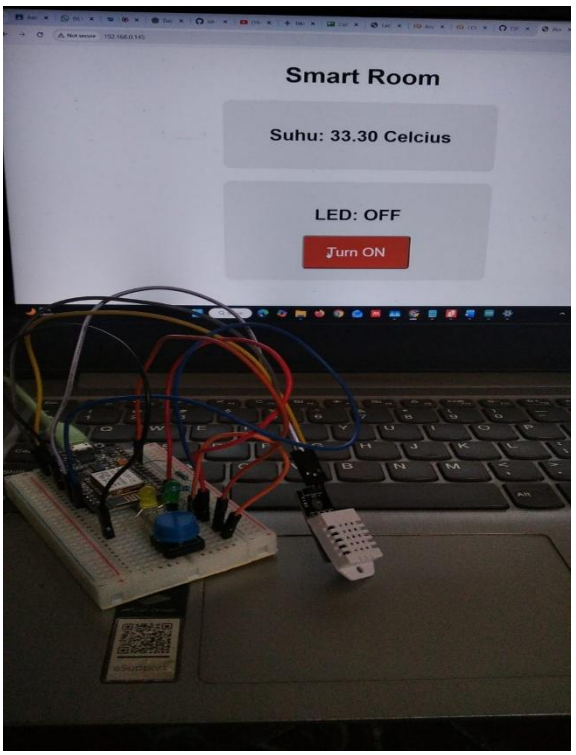
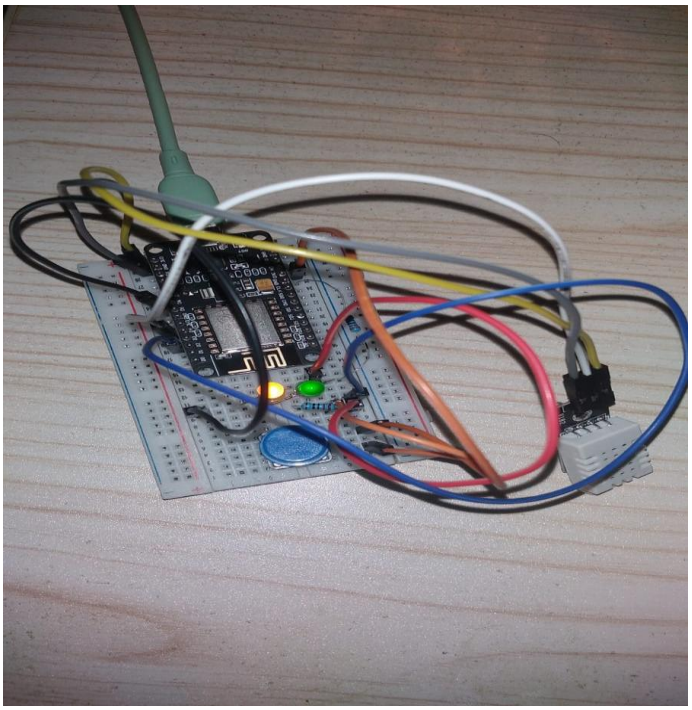
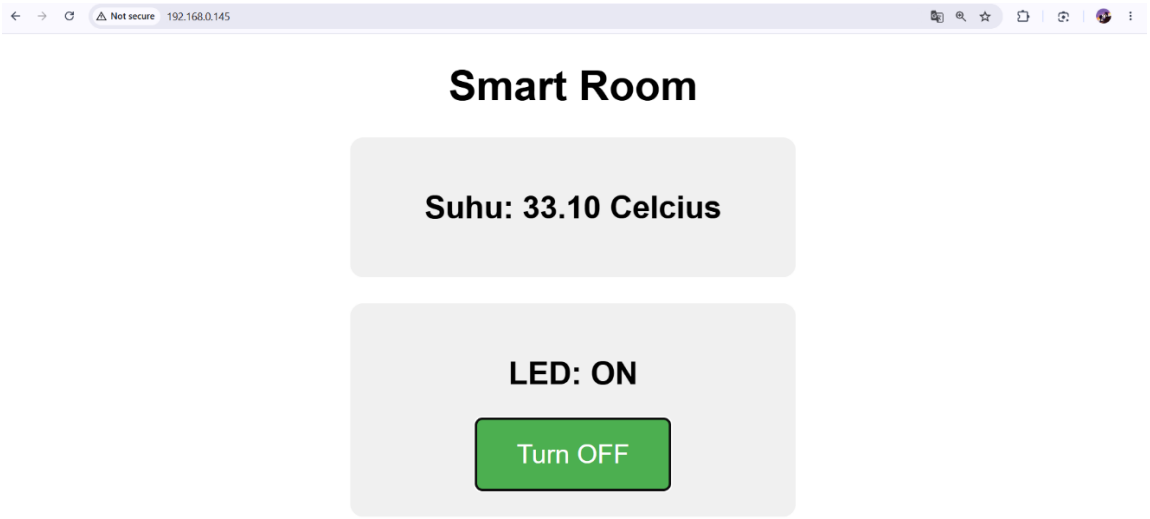


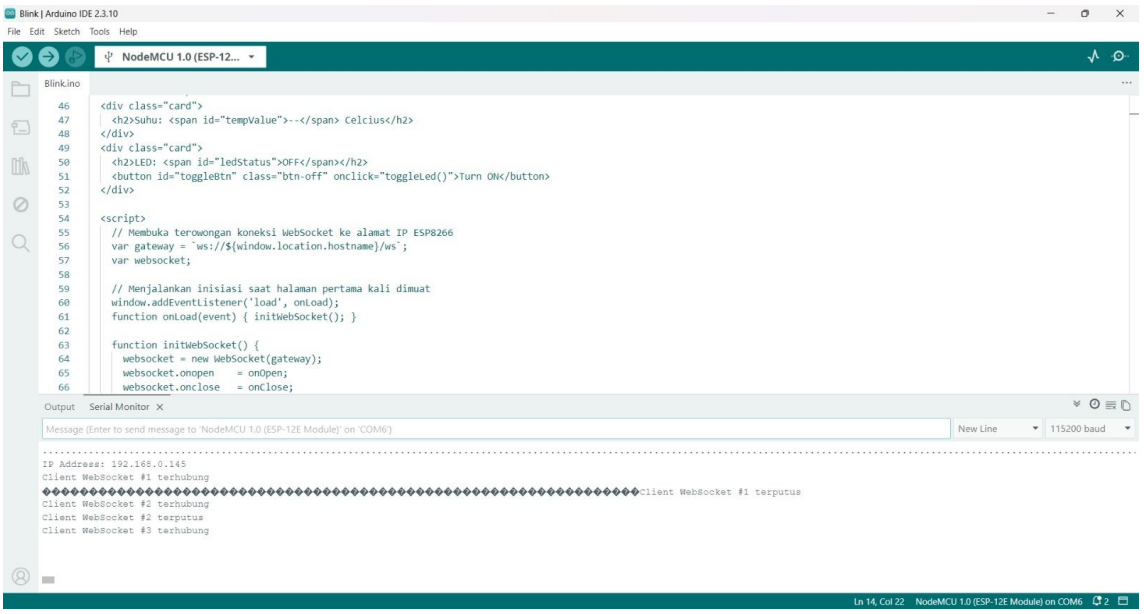
Foto alat



Tampilan web



Output dan code



Code :

```
#include <ESP8266WiFi.h>
#include <ESPAsyncTCP.h>
#include <ESPAsyncWebServer.h>
#include <DHT.h>

const char* ssid = "Network_eng";
const char* password = "d585de2865";

// Konfigurasi Pin
const byte dhtPin = 2;         // D4 (GPIO 2)
```

```
const byte buttonPin = 4;     // D2 (GPIO 4)
const byte ledPin = 12;       // D6 (GPIO 12)

DHT dht(dhtPin, DHT22);

// Variabel Pelacak Status (State & Cache)
bool ledState = false;
String currentTemp = "--";
int buttonState;
int lastButtonState = LOW;
unsigned long lastDebounceTime = 0;
const unsigned long debounceDelay = 50;
unsigned long lastTime = 0;

// Inisialisasi Async Web Server (port 80) & WebSocket (rute /ws)
AsyncWebServer server(80);
AsyncWebSocket ws("/ws");

// ---------------- HTML & JAVASCRIPT (FRONT-END) ----------------
const char index_html[] PROGMEM = R"rawliteral(
<!DOCTYPE html>
<html>
<head>
  <meta name="viewport" content="width=device-width, initial-scale=1">
  <title>Real-Time IoT Web</title>
  <style>
    body { font-family: Arial; text-align: center; }
    .card { background: #f0f0f0; margin: 20px auto; padding: 20px; max-
width: 300px; border-radius: 10px; }
    button { padding: 15px 30px; font-size: 20px; border-radius: 5px;
cursor: pointer; color: white;}
    .btn-on { background-color: #4CAF50; }
    .btn-off { background-color: #f44336; }
  </style>
</head>
<body>
  <h1>Smart Room</h1>
  <div class="card">
    <h2>Suhu: <span id="tempValue">--</span> Celcius</h2>
  </div>
  <div class="card">
    <h2>LED: <span id="ledStatus">OFF</span></h2>
    <button id="toggleBtn" class="btn-off" onclick="toggleLed()">Turn
ON</button>
  </div>

  <script>
```

```
// Membuka terowongan koneksi WebSocket ke alamat IP ESP8266
var gateway = `ws://${window.location.hostname}/ws`;
var websocket;

// Menjalankan inisiasi saat halaman pertama kali dimuat
window.addEventListener('load', onLoad);
function onLoad(event) { initWebSocket(); }

function initWebSocket() {
  websocket = new WebSocket(gateway);
  websocket.onopen    = onOpen;
  websocket.onclose   = onClose;
  websocket.onmessage = onMessage;
}

function onOpen(event) { console.log('WebSocket Terkoneksi'); }
function onClose(event) { setTimeout(initWebSocket, 2000); }

// Dipanggil saat tombol di layar web ditekan
function toggleLed(){
  websocket.send('toggle');
}

// Menangkap paket data JSON yang "didorong" (PUSH) oleh C++ NodeMCU
function onMessage(event) {
  var dataObj = JSON.parse(event.data);

  // Menyuntikkan teks angka Suhu ke dalam id "tempValue" HTML
  if(dataObj.suhu !== undefined) {
     document.getElementById('tempValue').innerHTML = dataObj.suhu;
  }

  // Merombak UI Tombol & Teks sesuai status hardware
  if(dataObj.led !== undefined) {
     var btn = document.getElementById('toggleBtn');
     var status = document.getElementById('ledStatus');
     if(dataObj.led == "1"){
       status.innerHTML = "ON";
       btn.innerHTML = "Turn OFF";
       btn.className = "btn-on";
     } else {
       status.innerHTML = "OFF";
       btn.innerHTML = "Turn ON";
       btn.className = "btn-off";
     }
  }
}
```

```
  </script>
</body>
</html>
)rawliteral";

// ---------------- BACK-END & WEBSOCKET LOGIC ----------------

// Fungsi menyebarkan paket data JSON ke SELURUH browser pengunjung
void notifyClients() {
  String jsonString = "{\"led\":\"" + String(ledState ? 1 : 0) + "\", ";
  jsonString += "\"suhu\":\"" + currentTemp + "\"}";
  ws.textAll(jsonString);
}

// Handler pesan masuk (dari klik browser)
void handleWebSocketMessage(void *arg, uint8_t *data, size_t len) {
  AwsFrameInfo *info = (AwsFrameInfo*)arg;
  if (info->final && info->index == 0 && info->len == len && info->opcode
== WS_TEXT) {
    data[len] = 0;
    if (strcmp((char*)data, "toggle") == 0) {
      ledState = !ledState;
      notifyClients();
    }
  }
}

// Event handler WebSocket bawaan AsyncWebServer
void onEvent(AsyncWebSocket *server, AsyncWebSocketClient *client,
AwsEventType type,
             void *arg, uint8_t *data, size_t len) {
  switch (type) {
    case WS_EVT_CONNECT:
      Serial.printf("Client WebSocket #%u terhubung\n", client->id());
      notifyClients(); // Kirim status terkini kepada klien yang baru
buka web
      break;
    case WS_EVT_DISCONNECT:
      Serial.printf("Client WebSocket #%u terputus\n", client->id());
      break;
    case WS_EVT_DATA:
      handleWebSocketMessage(arg, data, len);
      break;
  }
}

void setup() {
```

```
  Serial.begin(115200);

  // Mengatur pin tombol sebagai input (dengan resistor pull-down
eksternal)
  pinMode(buttonPin, INPUT);

  pinMode(ledPin, OUTPUT);
  digitalWrite(ledPin, LOW);
  dht.begin();

  WiFi.mode(WIFI_STA);
  WiFi.begin(ssid, password);
  while (WiFi.status() != WL_CONNECTED) { delay(500); Serial.print(".");
}
  Serial.println("\nIP Address: " + WiFi.localIP().toString());

  server.on("/", HTTP_GET, [](AsyncWebServerRequest *request){
    request->send_P(200, "text/html", index_html);
  });

  // Ikat handler WebSockets ke server utama
  ws.onEvent(onEvent);
  server.addHandler(&ws);
  server.begin();
}

void loop() {
  ws.cleanupClients();

  // 1. Eksekusi perangkat keras LED
  digitalWrite(ledPin, ledState ? HIGH : LOW);

  // 2. Baca Tombol Fisik (Debounce) tanpa nge-freeze program
  int reading = digitalRead(buttonPin);
  if (reading != lastButtonState) {
    lastDebounceTime = millis();
  }
  if ((millis() - lastDebounceTime) > debounceDelay) {
    if (reading != buttonState) {
      buttonState = reading;

      // Karena menggunakan resistor PULL-DOWN eksternal, tombol ditekan
= HIGH
      if (buttonState == HIGH) {
        ledState = !ledState;
        notifyClients(); // PUSH DATA KE BROWSER SEKETIKA
      }
```

```
    }
  }
  lastButtonState = reading;

  // 3. Baca DHT setiap 3 detik di memori cache, tanpa memblokir CPU
  if ((millis() - lastTime) > 3000) {
    float t = dht.readTemperature();
    if(!isnan(t)) {
      currentTemp = String(t);
      notifyClients(); // Push pembaruan suhu otomatis
    }
    lastTime = millis();
  }
}
```

Penjelasan Singkat Kode Khusus:

- ws.cleanupClients(): (Berada di siklus loop()). Fungsi penting dari pustaka *Asynchronous* ini bertugas membersihkan dan menutup sesi (*resource memori*) dari klien/*browser* yang sudah terputus atau menutup tab layarnya. Tanpa baris ini, memori RAM ESP8266 akan penuh (*memory leak*) dan berujung pada *Crash/Restart* jika terlalu banyak perangkat keluar-masuk membuka halaman *Dashboard*.
- pinMode(buttonPin, INPUT) dan if (buttonState == HIGH): Mempertahankan konfigurasi resistor *Pull-Down* eksternal 10k Ohm ke jalur GND (seperti pada Modul 1). Saat tombol dilepas, pin D2 membaca logika **LOW** (0V). Saat tombol ditekan, arus 3.3V mengalir sehingga terbaca logika **HIGH** yang memicu pembalikan status (*toggle*) LED secara instan.
- if ((millis() - lastTime) > 3000): Pemrograman *non-blocking*. Alih-alih membuat CPU tidur selama 3 detik menggunakan delay(3000) (yang akan membekukan respons WebSocket), kita menggunakan selisih penghitung jam internal CPU (millis()). Dengan demikian, CPU dapat terus siaga mengawasi sentuhan tombol fisik dan interaksi jaringan internet dengan kecepatan tinggi.
- var dataObj = JSON.parse(event.data): Sintaks JavaScript yang membedah teks kaku kiriman C++ berformat {"suhu": "30", "led": "1"} menjadi struktur data objek dinamis (*Dictionary*) untuk memperbarui elemen desain HTML dengan instan (*DOM Manipulation*).
- ws.textAll(): Perintah penyiaran (*broadcasting*) ampuh yang menembakkan/menyebarkan data secara seketika ke *seluruh* perangkat *browser* pengunjung (laptop, HP) secara bersamaan.

4.6 Latihan

- **Latihan 1 (Teori Jaringan):** Jelaskan dengan kata-kata Anda sendiri, mengapa teknologi koneksi *WebSockets* jauh lebih hemat konsumsi data/bandwidth internet dan tidak membuat layar berkedip jika dibandingkan dengan metode *Meta Refresh HTTP* HTML yang Anda gunakan pada Tugas Praktikum Modul 3?

Jawaban:

WebSockets jauh lebih hemat bandwidth karena menggunakan komunikasi dua arah (full-duplex) berkesinambungan melalui satu jalur TCP yang terus terbuka setelah handshake awal. Berbeda dengan Meta Refresh HTTP yang harus mengirim ulang header dan seluruh kode dokumen HTML/CSS secara terus-menerus setiap kali reload, WebSockets hanya mengirimkan data mentah berukuran sangat kecil (seperti JSON berisi status angka/teks saja). Layar juga tidak berkedip (flicker) karena proses pembaruan data dilakukan oleh JavaScript secara dinamis di latar belakang dengan memanipulasi DOM HTML (Document Object Model), tanpa memuat ulang seluruh struktur halaman web.

- **Latihan 2 (Analisis Waktu/Timer):** Apa bahaya sistematis yang akan terjadi pada komunikasi *WebSockets* (reaksi antara *browser* dan NodeMCU) jika Anda bersikeras nekat memanggil sintaks delay(3000) di dalam siklus fungsi void loop() pada draf arsitektur kompleks ini?

Jawaban:

Memakai delay(3000) di loop() bakal bikin NodeMCU "pingsan" (berhenti berproses) selama 3 detik.

Efek buruknya:

1. Koneksi WebSocket Putus (Timeout): NodeMCU nggak bisa kirim sinyal ping/pong, jadi browser menganggap koneksinya terputus (disconnect) dan bakal terus-menerus coba reconnect.
2. Respon Tombol Lag Parah: Klik di web atau penekanan push button fisik bakal delay sampai 3 detik, bahkan perintahnya bisa hilang/nggak terbaca sama sekali.

- **Latihan 3 (Modifikasi Front-End):** Analisislah fungsi JavaScript pada blok // Merombak UI Tombol & Teks sesuai status hardware. Nama ID elemen HTML () apa saja yang menjadi target injeksi/pengubahan teks oleh *script* yang ditulis oleh Anda?

Jawaban:

Target pengubahan teks oleh JavaScript pada kode saya ada pada 3 ID elemen HTML berikut:

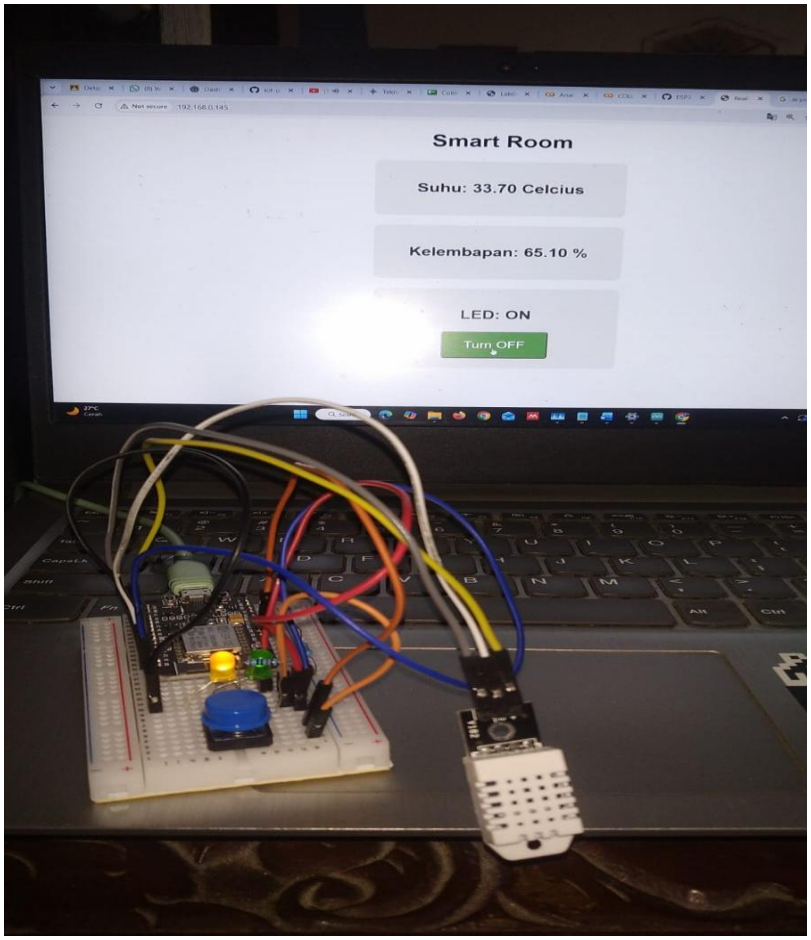
- tempValue: Untuk menyuntikkan dan memperbarui teks nilai suhu.
- ledStatus: Untuk mengubah teks status LED (menjadi ON atau OFF).
- toggleBtn: Untuk merombak label teks tombol (menjadi Turn ON atau Turn OFF) beserta tampilan warnanya.

- **Latihan 4 (Pengembangan Telemetri Sensor - Kelembapan):** Pada Praktikum 1, antarmuka web hanya menyajikan data suhu (*temperature*). Modifikasilah program agar sistem dapat membaca dan menampilkan data kelembapan (*humidity*) secara *real-time* via WebSockets:

    i. **Sisi Back-End (C++):** Buat variabel penampung (misal: String currentHum = "--";). Baca nilai kelembapan menggunakan fungsi dht.readHumidity() di dalam siklus timer *non-blocking* millis(). Perbarui fungsi notifyClients() agar menyisipkan data kelembapan ke dalam paket JSON yang dikirim ke *browser* (contoh format: {"led":"...", "suhu":"...", "hum":"..."}).

    ii. **Sisi Front-End (HTML & JavaScript):** Tambahkan elemen kartu ( ) baru pada kerangka HTML sebagai penampil kelembapan (misal: <h2>Kelembapan: -- %</h2>). Kemudian, modifikasi fungsi JavaScript onMessage(event) agar mengekstrak data kelembapan dari objek JSON dan memperbarui teks pada elemen humValue.
    Tuliskan baris kode modifikasi Anda (pada sisi HTML, JavaScript, dan C++)!

Jawaban:



Codenya:

```
#include <ESP8266WiFi.h>
#include <ESPAsyncTCP.h>
#include <ESPAsyncWebServer.h>
#include <DHT.h>

const char* ssid = "Network_eng";
const char* password = "d585de2865";

// Konfigurasi Pin
const byte dhtPin = 2;         // D4 (GPIO 2)
const byte buttonPin = 4;      // D2 (GPIO 4)
const byte ledPin = 12;        // D6 (GPIO 12)

DHT dht(dhtPin, DHT22);

// Variabel Pelacak Status (State & Cache)
bool ledState = false;
String currentTemp = "--";
String currentHum = "--";      // Variable penampung kelembapan
int buttonState;
int lastButtonState = LOW;
unsigned long lastDebounceTime = 0;
const unsigned long debounceDelay = 50;
unsigned long lastTime = 0;

// Inisialisasi Async Web Server (port 80) & WebSocket (rute /ws)
AsyncWebServer server(80);
AsyncWebSocket ws("/ws");

// ---------------- HTML & JAVASCRIPT (FRONT-END) ----------------
const char index_html[] PROGMEM = R"rawliteral(
<!DOCTYPE html>
<html>
<head>
  <meta name="viewport" content="width=device-width, initial-scale=1">
  <title>Real-Time IoT Web</title>
  <style>
    body { font-family: Arial; text-align: center; }
    .card { background: #f0f0f0; margin: 20px auto; padding: 20px; max-
width: 300px; border-radius: 10px; }
    button { padding: 15px 30px; font-size: 20px; border-radius: 5px;
cursor: pointer; color: white;}
    .btn-on { background-color: #4CAF50; }
    .btn-off { background-color: #f44336; }
  </style>
```

```
</head>
<body>
  <h1>Smart Room</h1>
  <div class="card">
    <h2>Suhu: <span id="tempValue">--</span> Celcius</h2>
  </div>

  <!-- Elemen Kartu Baru untuk Kelembapan -->
  <div class="card">
    <h2>Kelembapan: <span id="humValue">--</span> %</h2>
  </div>

  <div class="card">
    <h2>LED: <span id="ledStatus">OFF</span></h2>
    <button id="toggleBtn" class="btn-off" onclick="toggleLed()">Turn
ON</button>
  </div>

  <script>
    var gateway = `ws://${window.location.hostname}/ws`;
    var websocket;

    window.addEventListener('load', onLoad);
    function onLoad(event) { initWebSocket(); }

    function initWebSocket() {
      websocket = new WebSocket(gateway);
      websocket.onopen    = onOpen;
      websocket.onclose   = onClose;
      websocket.onmessage = onMessage;
    }

    function onOpen(event) { console.log('WebSocket Terkoneksi'); }
    function onClose(event) { setTimeout(initWebSocket, 2000); }

    function toggleLed(){
      websocket.send('toggle');
    }

    function onMessage(event) {
      var dataObj = JSON.parse(event.data);

      if(dataObj.suhu !== undefined) {
         document.getElementById('tempValue').innerHTML = dataObj.suhu;
      }

      // Injeksi data kelembapan ke elemen HTML humValue
```

```
      if(dataObj.hum !== undefined) {
         document.getElementById('humValue').innerHTML = dataObj.hum;
      }

      if(dataObj.led !== undefined) {
         var btn = document.getElementById('toggleBtn');
         var status = document.getElementById('ledStatus');
         if(dataObj.led == "1"){
           status.innerHTML = "ON";
           btn.innerHTML = "Turn OFF";
           btn.className = "btn-on";
         } else {
           status.innerHTML = "OFF";
           btn.innerHTML = "Turn ON";
           btn.className = "btn-off";
         }
      }
    }
  </script>
</body>
</html>
)rawliteral";

// ---------------- BACK-END & WEBSOCKET LOGIC ----------------

// Fungsi menyebarkan paket data JSON (Suhu, Kelembapan, LED) ke browser
void notifyClients() {
  String jsonString = "{\"led\":\"" + String(ledState ? 1 : 0) + "\", ";
  jsonString += "\"suhu\":\"" + currentTemp + "\", ";
  jsonString += "\"hum\":\"" + currentHum + "\"}";
  ws.textAll(jsonString);
}

void handleWebSocketMessage(void *arg, uint8_t *data, size_t len) {
  AwsFrameInfo *info = (AwsFrameInfo*)arg;
  if (info->final && info->index == 0 && info->len == len && info->opcode
== WS_TEXT) {
    data[len] = 0;
    if (strcmp((char*)data, "toggle") == 0) {
      ledState = !ledState;
      notifyClients();
    }
  }
}

void onEvent(AsyncWebSocket *server, AsyncWebSocketClient *client,
AwsEventType type,
```

```
             void *arg, uint8_t *data, size_t len) {
  switch (type) {
    case WS_EVT_CONNECT:
      Serial.printf("Client WebSocket #%u terhubung\n", client->id());
      notifyClients();
      break;
    case WS_EVT_DISCONNECT:
      Serial.printf("Client WebSocket #%u terputus\n", client->id());
      break;
    case WS_EVT_DATA:
      handleWebSocketMessage(arg, data, len);
      break;
  }
}

void setup() {
  Serial.begin(115200);

  pinMode(buttonPin, INPUT);
  pinMode(ledPin, OUTPUT);
  digitalWrite(ledPin, LOW);
  dht.begin();

  WiFi.mode(WIFI_STA);
  WiFi.begin(ssid, password);
  while (WiFi.status() != WL_CONNECTED) { delay(500); Serial.print(".");
}
  Serial.println("\nIP Address: " + WiFi.localIP().toString());

  server.on("/", HTTP_GET, [](AsyncWebServerRequest *request){
    request->send_P(200, "text/html", index_html);
  });

  ws.onEvent(onEvent);
  server.addHandler(&ws);
  server.begin();
}

void loop() {
  ws.cleanupClients();

  digitalWrite(ledPin, ledState ? HIGH : LOW);

  // Baca Tombol Fisik
  int reading = digitalRead(buttonPin);
  if (reading != lastButtonState) {
    lastDebounceTime = millis();
```

```
  }
  if ((millis() - lastDebounceTime) > debounceDelay) {
    if (reading != buttonState) {
      buttonState = reading;
      if (buttonState == HIGH) {
        ledState = !ledState;
        notifyClients();
      }
    }
  }
  lastButtonState = reading;

  // Baca Suhu & Kelembapan dari DHT11 setiap 3 detik
  if ((millis() - lastTime) > 3000) {
    float t = dht.readTemperature();
    float h = dht.readHumidity();

    if(!isnan(t) && !isnan(h)) {
      currentTemp = String(t);
      currentHum = String(h);
      notifyClients();
    }
    lastTime = millis();
  }
}
```